Hilbert-Schmidt Independence Criterion (HSIC) [6] to determine statistical dependence between two sets of variables.

$$CKA(X,Y) = \frac{HSIC(X,Y)}{\sqrt{HSIC(X,X)HSIC(Y,Y)}} \tag{3}$$

where HSIC between a pair of $N \times N$ matrices is defined in Equation 4

$$HSIC(X,Y) = \frac{trace(HXHHYH)}{(N-1)^2} \tag{4}$$

where H is a Centering matrix given by $H = I - \frac{1}{N}\mathbf{1}\mathbf{1}^T$.

2.3 Meta Algorithm

Our goal is to take a matrix of neural activations $X \in \mathbf{R}^{d_1 \times N}$, where d_1 is the number of neurons in the given layer and N is the number of examples for which we obtain the activations, and construct an affinity matrix $C_X \in \mathbf{R}^{N \times N}$, via Sparse Subspace Clustering [5] , where each non diagonal entry (i, j) of C_X denotes the affinity between input samples i and j. This gives us the ability to compare 2 matrices of neural activations, say $X \in \mathbf{R}^{d_1 \times N}$ and $Y \in \mathbf{R}^{d_2 \times N}$ by representing them as 2 Affinity Matrices $\in \mathbf{R}^{N \times N}$, namely C_X and C_Y and then using Centered Kernel Alignment[4] to compare the similarity of the 2 Affinity Matrices. This procedure helps us compare 2 different layers of a network, or 2 different layers of 2 architecturally different neural networks. We would like to point out that $C_X = |C| + |C^T|$ where the matrix C is obtained from Algorithm 1, where $|C|$ represents the absolute value function applied element-wise to C.

Algorithm 1: Matrix LASSO Minimization by ADMM

Data: Data Matrix X
Result: Sparse Representation C
initialization: $C^0 = \mathbf{0}$, $\Lambda_2^0 = \mathbf{0}$, $\mu_2 > 0$;
while *not converged* **do**
  $Z^{k+1} = (\tau X^T X + \mu_2 I)^{-1}(\tau X^T X + \mu_2(C^k - \frac{\Lambda_2^k}{\mu_2}))$;
  $C^{k+1} = \mathbf{S}_{\frac{1}{\mu_2}}(Z^{k+1} + \frac{\Lambda_2^k}{\mu_2})$; $\mathbf{S}$: Shrinkage operator
  $C^{k+1} = C^{K+1} - diag(C^{k+1})$;
  $\Lambda_2^{k+1} = \Lambda_2^k + \mu_2(Z^{K+1} - C^{k+1})$;
end

3 Problem and Experimental Setup

In this paper, we experiment with VGGs [22], ResNets [8], Wide-ResNets [24] and DenseNets [9] trained on CIFAR-10 and CIFAR-100 datasets [11]. Our approach

relies upon having access to activations of various hidden layers in the network for each input instance. As an example, for the first set of experiments in this study, we take outputs from a few pooling layers at the end of each block of a DenseNet to study the community structure of its subspace and also to learn the layer-wise dynamics of training progression. In another setting, we take focus on all the convolutional layers in the network and get their activations for an input.

3.1 Problem Formulation

We attempt to interpret and analyze neural networks by taking a matrix of its activations (layer-wise) $X \in \mathbf{R}^{d \times N}$, where d is the number of neurons in the subject layer and N is the number of inputs used for analysis. We first aim to learn a connectivity graph between these inputs based on their affinity scores obtained from SSC. Then we use this connectivity matrix to perform analysis pertaining to the community structure of the graph in Section 4, and in Section 6 we analyze instance based neighbourhoods of various inputs to better reconcile network predictions with the need to human oriented explanations

3.2 Experimental Details

All networks, unless otherwise stated, were trained with a learning rate of 0.1 and a weight decay of 5×10^{-4} with a learning rate step multiplier of 0.2 applied after every 30 epochs. All networks were trained for 100 epochs since that was sufficient to achieve optimal performance, with the exception of VGG-29, which required 160 epochs. For SSC computations, both τ and μ were set to 10, with μ being adaptive based on equation 3.13 in [3]. We recommend choosing τ and μ between 10 and 100. Our implementation is publicly available on GitHub [1].

4 Analysis of Network Training Dynamics

In this section, we analyze behaviour of the network as its training progresses. To demonstrate the training dynamics, we train the following networks[2] - ResNet [8], Wide ResNet [24] and DenseNet [9] on the datasets CIFAR-10 and CIFAR-100 [11]. Both ResNets and DenseNets are constructed by stacking and joining different residual blocks with multiple convolution layers residing in each block. For brevity and scalability of this experiment, we use the output of residual blocks instead of every convolution layer and the final classification layers. For a given combination of the network and the dataset, we present three results - layer-wise modularity of the sparse subspace affinity graphs, SSC-CKA based layer-wise similarity to understand training dynamics, and analogous training dynamic analysis with Linear-CKA.

[1] URL - `https://github.com/23Uday/Subspace-Clustering-based-analysis-of-Neural-Networks`

[2] Networks used from: `https://github.com/kuangliu/pytorch-cifar` and `https://github.com/meliketoy/wide-resnet.pytorch`

4.1 Community Structure via Graph Modularity

We analyze the community structure of the sparse subspace affinity graph for various layers at each epoch. We do this by calculating the modularity [16] of the learned subspace affinity graph. Modularity is a measure of the structure of a graph, measuring the density of connections within a module or community. High modularity in a graph indicates dense connectivity between nodes of the same type, while low modularity indicates dense connectivity between nodes of different types, where a node type is its class label. Mathematically, modularity of a graph can be represented as follows

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j) \tag{5}$$

where m is the number of edges in the graph, A_{ij} is the weight of the edge between nodes i and j, k_i is the degree of node i, c_i is the class label of node i and $\delta(c_i, c_j)$ is 1 if node i and node j are of the same class and 0 otherwise.

We present this analysis in Figure 1a and Figure 2a. We observe that as we go deeper in the network, the modularity of the learned sparse subspace affinity graph increases, implying that earlier layers of the network cluster examples of different classes together in the same subspace and deeper layers of the network tend to separate out classes into different disjoint subspaces. Another noteworthy observation is that the modularity scores of subspace affinity graph learned from earlier layers tends to saturate earlier in training when compared to the modularity of the subspace affinity graphs of deeper layers. This phenomenon is consistent across different architectures and different datasets. A similar observation regarding earlier saturation of shallower layers in training dynamics is also made in [20], albeit in context of representational similarity, which we address next.

4.2 Layer-wise Training Dynamics and Convergence

For our second analysis, we compute the Centered Kernel Alignment (CKA) scores between the sparse subspace affinity graph of a layer at a given epoch and the same layer after the final epoch. This allows us to observe the rates at which layers converge to their final states. These results are shown in Figure 1b and Figure 2b. We observe a similar pattern of shallower layers converging to their final representation much earlier, when compared to deeper layers of the network. Another key observation that we make pertains to the behaviour of representations when the learning rate of the optimiser is reduced to improve convergence. For training of all the networks, we start with a learning rate of 0.1 and reduce it by a factor of 0.2 after every 30 epochs using SGD with learning rate decay. At each step size decay, in addition to an improvement in network accuracy, we also observe a jump in the CKA similarity scores of the epoch's subspace affinity graph towards the CKA scores of final epoch's subspace affinity graph. We note that this jump is less prominent in shallower layers of the network